



Lab 10

Timer

Prof. Chih-Cheng Tseng (曾志成)

cctseng@mail.ntou.edu.tw

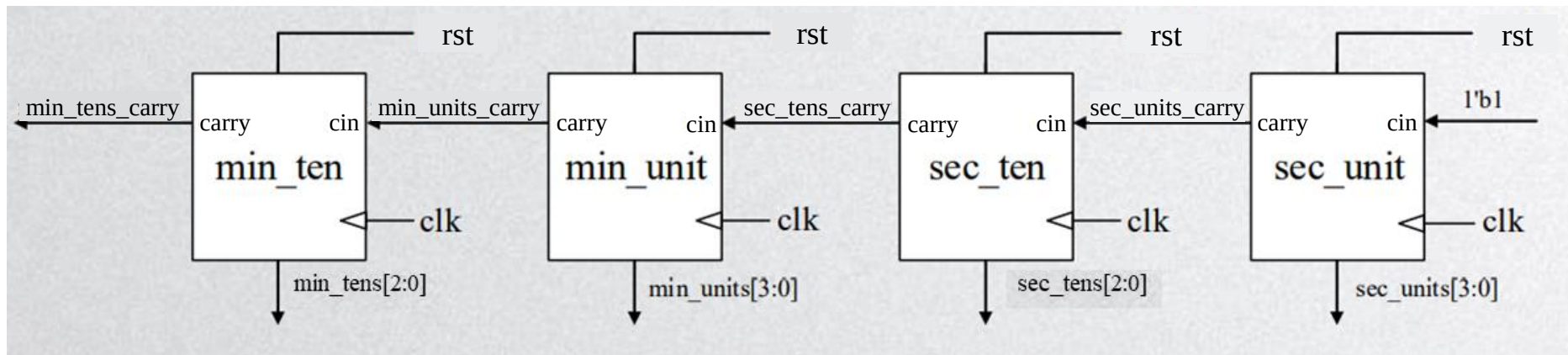
Contents of this slide are provided by NTOU VLSI Lab601

Objectives

- Design a timer 00:00 ~ 59:59 (mm:ss)
- Bonus: Design a clock 00:00 ~ 23:59 (hh:mm)

LAB – 00:00 ~ 59:59 Timer

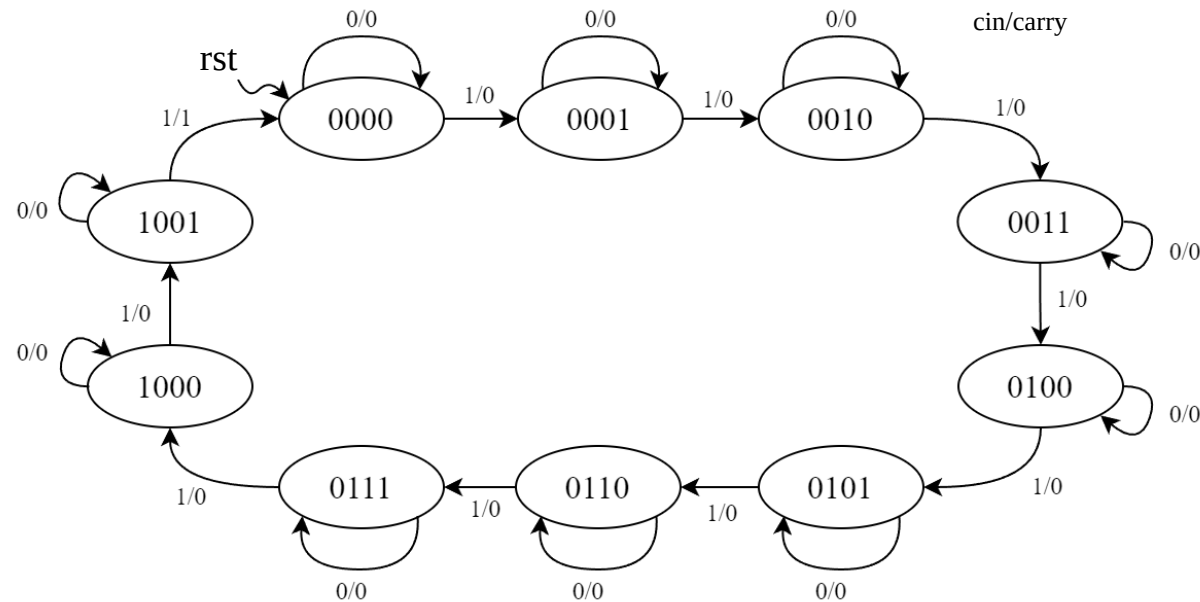
- Using the *D* FFs to design a 00:00 to 59:59 timer.
- Verify your design by the signal simulation and DE0 programming.
- Signal settings
 - Input: *clk* and *rst*
 - Output: *min_tens*[2:0], *sec_tens*[2:0], *min_units*[3:0], *sec_units*[3:0]
- Architecture diagram



LAB – 00:00 ~ 59:59 Timer

Design The Units Digit (個位數) (1/5)

- A counter composed of 4 *D* flip-flops counts from 0 to 9. When it reaches 9, it generates a carry (i.e., carry = 1) and resets to 0 on the next cycle.
- Signal setting
 - Input: *clk*, *rst*, and *cin*
 - Output: $q[3:0] = \{A, B, C, D\}$, *carry*
- State diagram



LAB – 00:00 ~ 59:59 Timer

Design The Units Digit (個位數) (2/5)

- State table

Current status (t)					Next status ($t + 1$)				carry
cin	$A(t)$	$B(t)$	$C(t)$	$D(t)$	$A(t + 1)$	$B(t + 1)$	$C(t + 1)$	$D(t + 1)$	$carry$
0	0	0	0	0	0	0	0	0	0
0	0	0	0	1	0	0	0	1	0
0	0	0	1	0	0	0	1	0	0
0	0	0	1	1	0	0	1	1	0
0	0	1	0	0	0	1	0	0	0
0	0	1	0	1	0	1	0	1	0
0	0	1	1	0	0	1	1	0	0
0	0	1	1	1	0	1	1	1	0
0	1	0	0	0	1	0	0	0	0
0	1	0	0	1	1	0	0	1	0
0	1	0	1	0	X	X	X	X	X
0	1	0	1	1	X	X	X	X	X
0	1	1	0	0	X	X	X	X	X
0	1	1	0	1	X	X	X	X	X
0	1	1	1	0	X	X	X	X	X
0	1	1	1	1	X	X	X	X	X

LAB – 00:00 ~ 59:59 Timer

Design The Units Digit (個位數) (3/5)

- State table

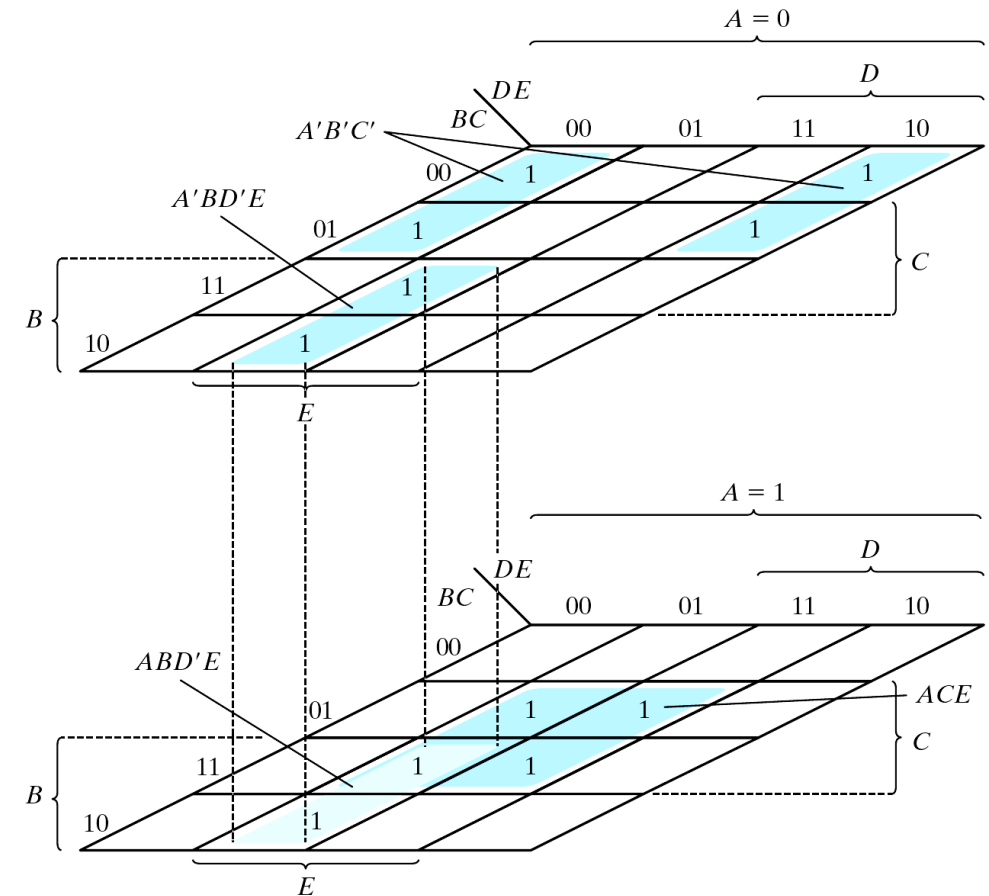
Current status (t)					Next status ($t + 1$)				carry
cin	$A(t)$	$B(t)$	$C(t)$	$D(t)$	$A(t + 1)$	$B(t + 1)$	$C(t + 1)$	$D(t + 1)$	$carry$
1	0	0	0	0	0	0	0	1	0
1	0	0	0	1	0	0	1	0	0
1	0	0	1	0	0	0	1	1	0
1	0	0	1	1	0	1	0	0	0
1	0	1	0	0	0	1	0	1	0
1	0	1	0	1	0	1	1	0	0
1	0	1	1	0	0	1	1	1	0
1	0	1	1	1	1	0	0	0	0
1	1	0	0	0	1	0	0	1	0
1	1	0	0	1	0	0	0	0	1
1	1	0	1	0	X	X	X	X	X
1	1	0	1	1	X	X	X	X	X
1	1	1	0	0	X	X	X	X	X
1	1	1	0	1	X	X	X	X	X
1	1	1	1	0	X	X	X	X	X
1	1	1	1	1	X	X	X	X	X

LAB – 00:00 ~ 59:59 Timer

Design The Units Digit (個位數) (4/5)

● Five-Variable K-Map

		$A = 0$						$A = 1$					
		DE		D				DE		D			
BC		00	01	11	10			00	01	11	10		
B	00	0	1	3	2	C		16	17	19	18	C	
	01	4	5	7	6			20	21	23	22		
	11	12	13	15	14			28	29	31	30		
	10	8	9	11	10			24	25	27	26		
		E						E					



LAB – 00:00 ~ 59:59 Timer

Design The Units Digit (個位數) (5/5)

- K-Map with 5 inputs for $A(t + 1)$ of the units digit

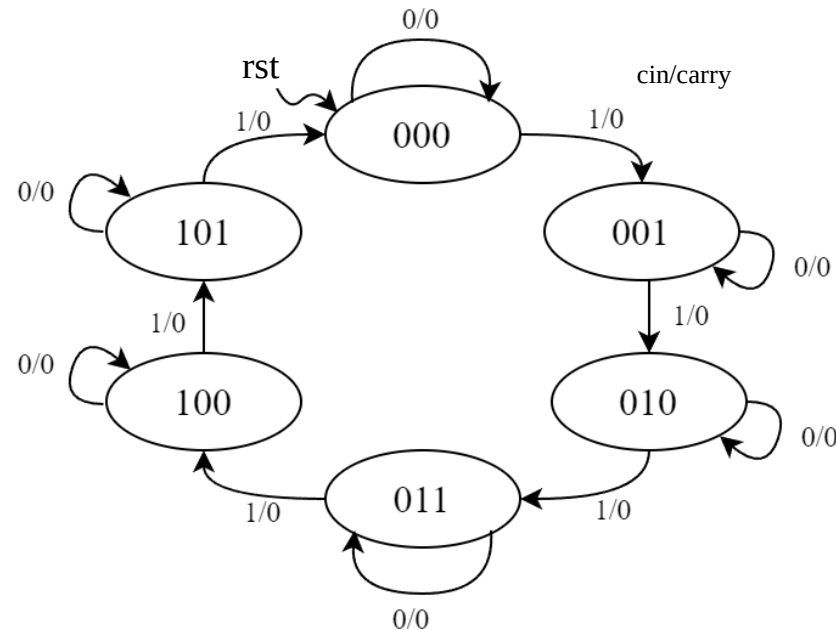
Cin ^{C D} A B	00	01	11	10
000	0	1	3	2
001	4	5	7	6
011	12	13	15	14
010	8	9	11	10
110	24	25	27	26
111	28	29	31	30
101	20	21	23	22
100	16	17	19	18

C(t) D(t) Cin A(t) B(t)	00	01	11	10
000	0	0	0	0
001	0	0	0	0
011	X	X	X	X
010	1	1	X	X
110	1	0	X	X
111	X	X	X	X
101	0	0	1	0
100	0	0	0	0

LAB – 00:00 ~ 59:59 Timer

Design The Tens Digit (十位數) (1/2)

- A counter composed of 3 *D* FFs counts from 0 to 5. When it reaches 5, it generates a carry (i.e., carry = 1) and resets to 0 on the next unit digit carry.
- Signal settings
 - Input: *clk*, *rst*, and *cin*
 - Output: $q[2:0] = \{A, B, C\}$, carry
- State diagram



LAB – 00:00 ~ 59:59 Timer

Design The Tens Digit (十位數) (2/2)

- State table

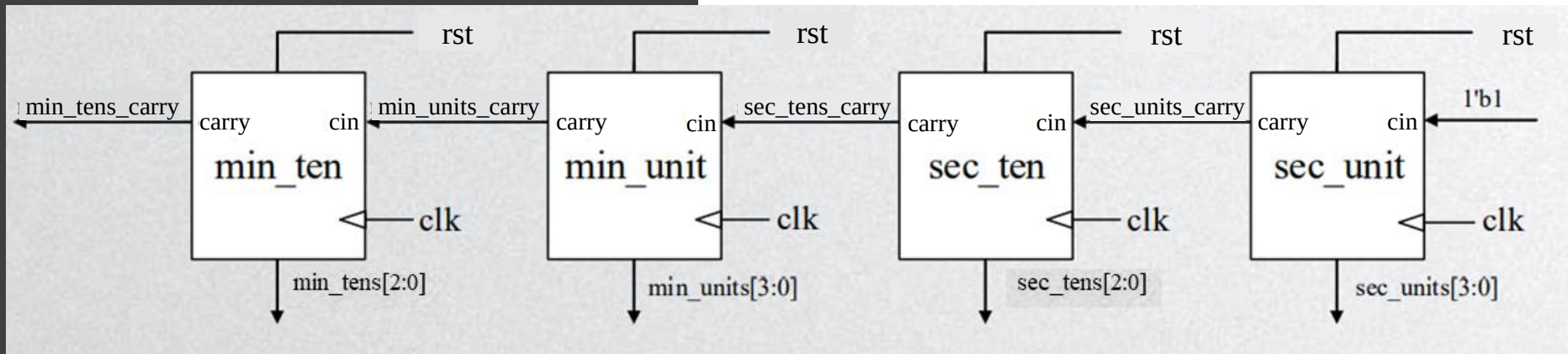
Current State (t)				Next State ($t + 1$)			carry
cin	$A(t)$	$B(t)$	$C(t)$	$A(t + 1)$	$B(t + 1)$	$C(t + 1)$	$carry$
0	0	0	0	0	0	0	0
0	0	0	1	0	0	1	0
0	0	1	0	0	1	0	0
0	0	1	1	0	1	1	0
0	1	0	0	1	0	0	0
0	1	0	1	1	0	1	0
0	1	1	0	X	X	X	X
0	1	1	1	X	X	X	X

Current State (t)				Next State ($t + 1$)			carry
cin	$A(t)$	$B(t)$	$C(t)$	$A(t + 1)$	$B(t + 1)$	$C(t + 1)$	$carry$
1	0	0	0	0	0	1	0
1	0	0	1	0	1	0	0
1	0	1	0	0	1	1	0
1	0	1	1	1	0	0	0
1	1	0	0	1	0	1	0
1	1	0	1	0	0	0	1
1	1	1	0	X	X	X	X
1	1	1	1	X	X	X	X

LAB – 00:00 ~ 59:59 Timer

Top-Level Code

```
module timer(  
    input clk, rst,  
    output [2:0] sec_tens, min_tens,  
    output [3:0] sec_units, min_units  
);  
  
    logic sec_units_carry, min_units_carry, sec_tens_carry, min_tens_carry;  
  
    unit_digit unit_digit_sec(  
        .clk(clk),  
        .rst(rst),  
        .cin(1'b1),  
        .q(sec_units),  
        .carry(sec_units_carry)  
    );  
  
    ten_digit ten_digit_sec(  
        // write by yourself  
    );  
  
    unit_digit unit_digit_min(  
        // write by yourself  
    );  
  
    ten_digit ten_digit_min(  
        // write by yourself  
    );  
endmodule
```



LAB – 00:00 ~ 59:59 Timer Testbench



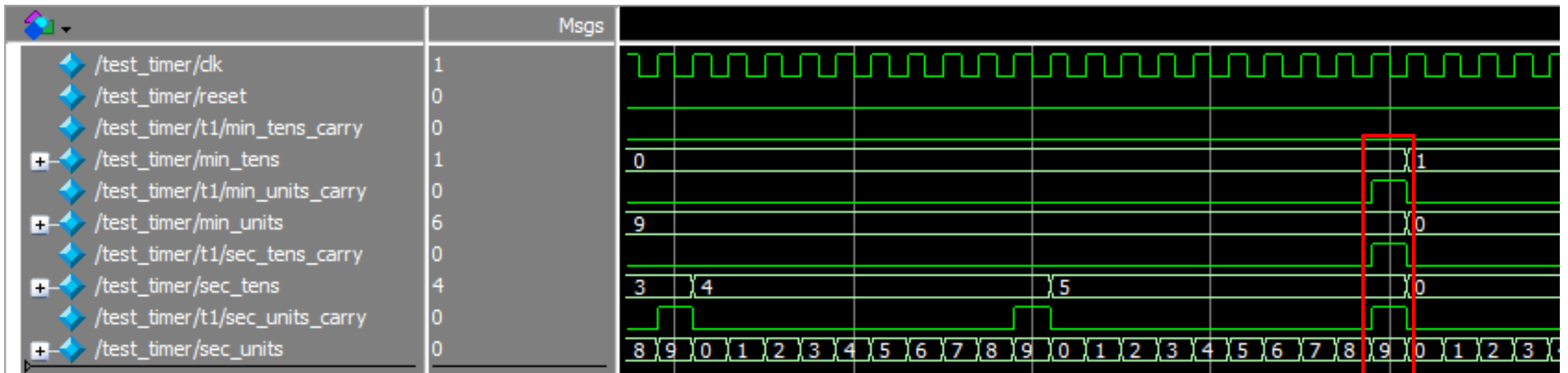
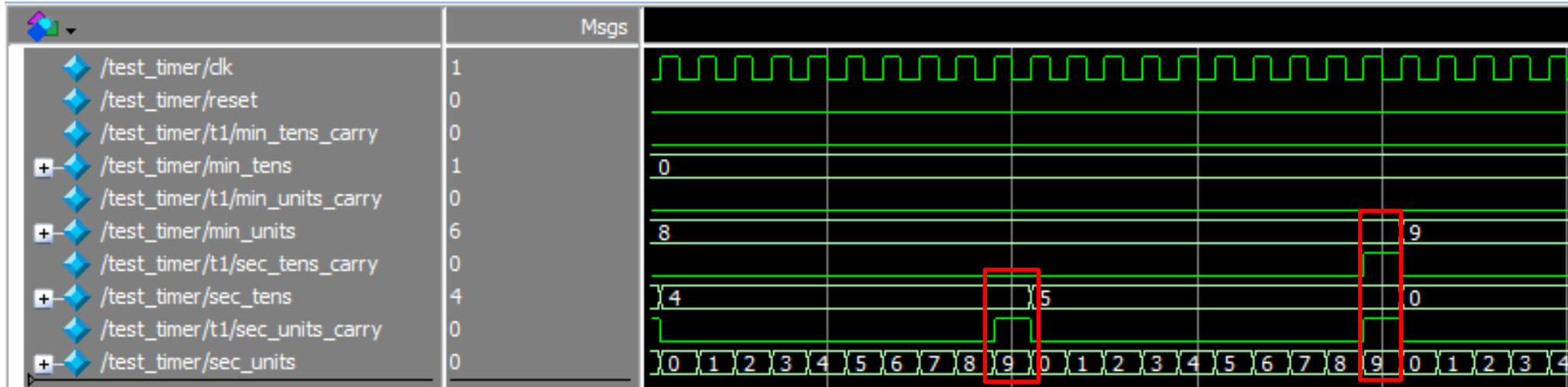
```
module test_timer;

    logic clk, rst;
    logic [2:0] sec_tens, min_tens;
    logic [3:0] sec_units, min_units;

    timer t1(
        // write by yourself
    );

    always #5 clk = ~clk;
    initial
    begin
        clk = 0; rst = 1;
        #10 rst = 0;
        #10000 $stop;
    end
endmodule
```

LAB – 00:00 ~ 59:59 Timer Waveform



LAB – 00:00 ~ 59:59 Timer

DE0 Pin Setting



Signals	Pin
<i>clk</i>	CLK
<i>rst</i>	~BNT[0]
<i>sec_unit</i>	HEX0
<i>sec_tens</i>	HEX1
<i>min_unit</i>	HEX2
<i>min_tens</i>	HEX3

Bonus – 00:00 ~ 23:59 Clock

- Using the *D* FFs, design a clock that can count from 00:00 up to 23:59. When hour_tens = 2 and hour_unit = 4, reset all the *D* FFs to zero.
- Signal settings
 - Input: *clk* and *rst*
 - Output: hour_tens[2:0], min_tens[2:0], hour_units[3:0], min_units[3:0]
- Architecture diagram

